Pràctica 2 - IA



Autors: Xavier Estades Novo i Joan Gelabert Oliver

Aquesta pràctica ha estat feta amb ajuda de ChatGPT 4 i DeepSeek en la creació de codi i en la revisió gramatical

Índex

1. Descripció del problema
2. Els algoritmes
    a. Q-Learning i Sarsa
    b. MonteCarlo
    c. Algoritme Genètic
    d. Programació Dinàmica
3. Explicació de la implementació dels algoritmes
4. Comparació de rendiments
5. Conclusió

Descripció del problema

El problema FrozenLake consisteix en un tauler en forma de graella on un agent ha de desplaçar-se des del punt d'inici fins a l'objectiu evitant caure en els forats, tot navegant per caselles de gel segur i afrontant la incertesa del terreny lliscant, que pot desviar aleatòriament els seus moviments.

Els algoritmes

Sarsa i Q-Learning

Característiques compartides

Hem triat Sarsa/Q-Learning perqué:

- El moviment es estocàstic: Quan l'agent decideix realitzar una acció, no sap a ciència certa si l'acabarà realitzant, degut a que el sól patina. La probabilitat de que l'agent acabi realitzant l'acció escollida es de ⅓ i ⅔ és la probabilitat que té l'agent de patinar als laterals.

Aquest comportament introdueix **incertesa en les transicions**, fa que **el mateix estat no sempre tengui el mateix resultat** i implica que una política òptima ha de tenir en compte **el risc de patinar**.

- L'espai d'estats és discret i manejable: És un taulell 4x4 amb accions ben definides(UP-3, DOWN-1, LEFT-0, RIGHT-2). Això vol dir que mètodes de valoració com Q(s,a) són perfectament aplicables.

La valoració Q(s,a) de Sarsa és:

$$Q(S_t,A_t) \leftarrow Q(S_t,A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1},A_{t+1}) - Q(S_t,A_t)]$$

La valoració Q(s,a) de Q-Learning és:

$$Q(S_t,A_t) \leftarrow Q(S_t,A_t) + \alpha[R_{t+1} + \gamma max_a Q(S_{t+1},A_{t+1}) - Q(S_t,A_t)]$$

Els métodes de valoració basats en *Q(s,a)* permeten als algortimes d'aprenentatge per reforç actualitzar el coneixement de l'agent sobre la convinença d'executar una acció a en un estat s. Aquest procés d'actualització s'executa després de cada transició observada i és el que fa possible que l'agent aprengui a partir de la seva pròpia experiència.

- Hi ha estats terminalment perillosos(els forats):Caure dins un forat mos condueix al final de l'episodi. Per tant, la sensibilitat al risc i la manera com cada algortime tracta transicions arriscades i la percepció de risc és clau.
- L'agent ha d'equilibrar exploració i explotació: Sense explorar, no se descobriran camins i, sense explotar, no convergirà

Característiques específiques

SARSA

Hem triat SARSA en concret perqué:

- És on-policy: Els algoritmes on-policy milloren la mateixa política que utilitzen per actuar sobre l'entorn. Això significa que aprenen el valor de la política realment executada per l'agent, incloent-hi les seves decisions d'exploració. Per aquest motiu, l'actualització dels valors Q(s,a) reflecteix fidelment el comportamente fectiu de l'agent i no només la política òptima teòrica.
- SARSA incorpora el risc real del terreny lliscant: Com que apren el valor de la trajectòria ε-greedy que està fent, té en compte la possibilitat:
    - de lliscar
    - de caure en un forat
    - de fer moviments no desitjats
- És adequat per entorns on una acció arriscada pot ser fatal: SARSA tendeix a evitar camins que passen massa aprop dels forats, perqué:
    - la política ε-greedy realment executa moviments aleatoris
    - SARSA ho incorpora al càlcul de Q
- Apren polítiques "risk-aware": SARSA prefereix anar per camins més llargs però amb estats més segurs

Q-Learning

Hem triat Q-Learning en concret perqué:

- És un algortime off-policy: Això ve a dir que aprèn la política òptima independentment de com explora. Tot i utilitzar ε-greedy per explorar, l'actualització de Q es fa assumint de l'agent sempre triarà l'acció òptima al següent estat.
- És robust davant el renou: Q-Learning fa una actualització basada en bootstrapping, i això li permet aprofitar immediatament nova informació i implementar ràpidament resultats inesperats provocats pel terreny lliscant.

MonteCarlo

Hem posat a MonteCarlo separat ja que funciona completament diferent a SARSA i Q-Learning ja que no actualitza després de cada transició, sinó després de l'episodi complet.

La valoració Q(s,a) de MonteCarlo és:

$$Q(s,a) = \textit{mitjana de tots els retorns observats per } (s,a)$$

- MonteCarlo no fa bootstrapping, sinó que apren dels retorns finals: El bootstrapping és el procés mitjançant el qual un algoritme actualitza el valor d'un estat o acció utilitzant estimacions prèvies, en lloc d'esperar a observar el retorn complet d'un

episodi. Dit d'una altre forma, un algoritme que fa bootstrapping s'autoactualitza utilizant els seus propis valors actuals de Q per predir el futur. **Estimes el futur amb el que ja tens**

- Funciona molt bé en entorns discrets i petits: El nombre d'estats i accions és reduït i els episodis són molt curts, cosa que permet obtenir el retorn complet ràpidament. A més, la recompensa binària (0 o 1) fa que les estimacions tinguin poca variància i siguin estables. Com que no utilitza bootstrapping, evita l'acumulació d'errors i es manté robust fins i tot amb polítiques molt exploratòries. Per tot això, convergeix de manera pràctica amb pocs episodis en aquest tipus d'entorns.

Algoritme Genètic

Hem triat l'Algorisme Genètic en concret perquè:

- **És un mètode de cerca directa a l'espai de polítiques:** A diferència de SARSA o Q-Learning, l'Algorisme Genètic no necessita estimar funcions de valor intermèdies ($Q(s,a)$) ni calcular errors de predicció temporal. En lloc d'això, evoluciona directament el "cromosoma" (la taula que mapeja cada estat a una acció). Això simplifica el problema, ja que en FrozenLake el genoma és molt compacte (només 16 gens, un per cada casella).
- **Gestiona l'estocasticitat (relliscades) a través de l'avaluació estadística (*Fitness*):** En un entorn amb relliscament, una bona acció pot tenir un mal resultat per mala sort. L'Algorisme Genètic supera això avaluant cada individu múltiples vegades o al llarg de generacions. Les polítiques que són "arriscades" (que passen massa a prop dels forats) acaben tenint una *fitness* mitjana baixa i són descartades per la selecció natural, mentre que les polítiques robustes que minimitzen el risc de caure sobreviuen i es reprodueixen.
- **L'exploració és global gràcies a la mutació:** Mentre que els algorismes de reforç clàssics depenen d'una exploració local (epsilon-greedy) que es va reduint, l'Algorisme Genètic manté la diversitat a la població mitjançant la mutació. Això evita que l'agent quedi atrapat fàcilment en òptims locals o en camins sub-òptims, permetent descobrir rutes alternatives que algorismes basats en gradient podrien ignorar.
- **No depèn de *bootstrapping* ni de cadenes de Markov:** Com que avalua l'episodi sencer (similar a MonteCarlo) per assignar una puntuació final, no pateix de la propagació d'errors d'estimació. L'agent simplement aprèn: "aquesta seqüència de gens (accions) m'ha portat a la meta més vegades que aquesta altra", sense necessitat d'entendre la dinàmica interna de les transicions.

Implementació dels algoritmes

1. Paràmetres inicials

   Els paràmetres utilitzats en l'entrenament són els següents:

   - Learning rate(α): 0,1 → El valor d'α controla quant de pes té la nova informació respecte a l'antiga en cada actualització de Q. Un α massa alt produiria aprenentatge inestable, especialment en un entorn estocàstic com FrozenLake i un α massa baix faria que l'agent aprenés massa lent, necessitant moltíssims d'episodis per aprendre. MonteCarlo no necessita α perqué no fa bootstrapping

     En un entorn petit, discret i amb recompenses binàries (0 ò 1), 0,1 és un valor equilibrat

   - Discount factor(γ):0,9 → El paràmetre γ determina com d'importants són les recompenses futures. Hem triat γ = 0,9 perqué esteim en un entorn on l'única recompensa positiva és al final, no hi ha recompenses intermèdies, γ massa baix faria que l'agent no valori suficientment arribar a l'objectiu i un γ massa alt gairebé no faria distinció entre trajectòries curtes i llargues
   - Exploració(ε): 0,1 → ε controla l'exploració, és a dir, amb quin percentatge de probabilitat l'agent fa una acció aleatòria. ε = 0,1 garanteix que l'agent explora un 10% de les vegades, sent un bon valor per els entorns petits i discrets. Si ε fos massa baix, el terreny patinant podria bloquejar la convergència cap a una bona política i si ε fos massa alt, Q-Learning s'inestabilitzaria ja que Q-Learning assumeix que el futur serà òptim, però si l'agent actua de manera gairebé aleatòria, aquesta suposició deixa de tenir sentit i començarien els problemes
   - Nombre d'episodis: 250000 → FrozenLake té episodis molt curts, però també té estats on la probabilitat d'arribar a l'objectiu és baixa. Per això necessitam un nombre alt d'episodis per assegurar el correcte maneig de l'efecte de patinar i poder comparar estadísticament els algoritmes amb resultats estables. Cal compensar la variabilitat amb quantitat

Aquests valors no són arbitraris, cada paràmetre controla un aspecte de l'aprenentatge i influeix en la convergència, l'estabilitat i el balanç entre exploració i explotació

PARAMETRES INICIALS ALGORITME GENÈTIC

- Mida de la població: 50 → Aquest valor determina la diversitat genètica inicial i la capacitat de l'algorisme per cobrir l'espai de solucions. Una població massa petita provocaria una convergència prematura cap a òptims locals (tots els individus serien iguals massa ràpid), mentre que una població massa gran incrementaria innecessàriament el cost computacional.
- Taxa de mutació: 0,01 (1%) → Aquest paràmetre controla l'exploració, introduint petites variacions aleatòries als "cromosomes" (accions). Un valor de 0,01 assegura que la majoria de l'estructura de la política guanyadora es manté intacta (explotació), però permet escapar d'estancaments.
- Episodis d'avaluació per individu: 50 → Aquest és el paràmetre crític per gestionar l'estocasticitat de is_slippery=True. A diferència dels entorns deterministes on basta una execució, aquí necessitam fer la mitjana de 50 intents per obtenir un fitness fiable. Un nombre massa baix d'episodis faria que individus "afortunats" (que han sobreviscut per atzar) fossin seleccionats erròniament, desestabilitzant l'evolució. Amb 50 episodis, filtram el renou estadístic i premiam la robustesa real.
- Generacions: 200 → Permet que l'algorisme tengui temps suficient per refinar la política mitjançant petits canvis incrementals. Atès que utilitzam elitisme (sempre guardam el millor), un nombre alt de generacions no presenta risc de degradació de la solució, només de cost temporal, i assegura que arribam a la millor combinació possible d'accions.

SARSA

2. Representació de Q

   Els valors d'acció s'han representat mitjançant una matriu NumPy de dimensions (n estats x n accions). Aquí un exemple de les tres primeres entrades de la matriu Q, on s són els estats i els índexs(0-3) de de la matriu representen les accions, sent 0(anar esquerra), 1(anar cap avall), 2(anar cap a la dreta) i 3(anar cap amunt):

   s=0 → [0.06 0.044 0.05 0.042]

   s=1 → [0.018 0.021 0.026 0.05]

   s=2 → [0.072 0.034 0.035 0.03]

3. Política ε-greedy

   Aquesta política permet un equilibri controlat entre explotació(triar l'acció amb el valor Q més alt fins el moment) i exploració(triar una acció aleatòria).

   - Amb probabilitat ε, se farà exploració
   - Amb probabilitat 1-ε, se farà explotació

Aquesta decisió és especialment rellevant en FrozenLake, on la incertesa del terreny lliscant requereix un cert grau d'exploració per evitar convergir a polítiques menys òptimes.

4. Regla d'actualització de SARSA

   La regla aplicada és l'actualització on-policy de SARSA:

$$Q(S_t,A_t) \leftarrow Q(S_t,A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1},A_{t+1}) - Q(S_t,A_t)]$$

La qual aprèn el valor de la política que realment executa l'agent, és més conservador en entorns arriscats i tendeix a produir polítiques més segures. També fa bootstrapping, és a dir, actualitza els valors Q utilitzant estimacions pròpies del futur.

Q-Learning

1. Representació de Q

   Igual que SARSA

2. Política ε-greedy

   Igual que SARSA

3. Regla d'actualització de Q-Learning

   La regla d'actualitzaci´p empleada és la següent:

$$Q(S_t,A_t) \leftarrow Q(S_t,A_t) + \alpha[R_{t+1} + \gamma max_a Q(S_{t+1},A_{t+1}) - Q(S_t,A_t)]$$

Aquesta regla presenta dues propietats claus:

- Bootstrapping: Actualitza els valors Q utilitzant estimacions pròpies del futur.
- Off-policy: Assumeix que l'agent triarà l'acció òptima en el futur

Això fa que Q-Learning produeixi polítiques més agressives i eficients.

4. Paràmetres inicials

   Igual que SARSA

MonteCarlo

1. Representació de Q

   Els valors de Q s'han representat amb un defaultdict. Aquesta decisió permet:

   - inicialitzar automàticament els valors Q de qualsevol estat que aparegui durant l'episodi
   - flexibilitat en entorns on alguns estats poden aparèixer rarament

2. Política ε-greedy

   Igual que SARSA

   Estratègia d'exploració

   L'exploració és constant amb un valor d'ε fixat per paràmetre. Això ajuda a:

   - estudiar l'impacte d'epsilon sobre la qualitat de la política
   - comparar-lo experimentalment amb Q-Learning i SARSA en condicions similars

3. Generació d'episodis complets

   El MonteCarlo genera episodis sencers abans d'actualitzar cap valor Q. Aquest enfocament:

   - Captura el retorn total sense dependència d'estimacions intermèdies
   - És coherent amb l'entorn FrozenLake, on els episodis són habitualment molt curts
   - Dona estimacions més estables que Temporal Difference (SARSA i Q-Learning) quan ε és alt o el terreny patina

Això es deu al fet que Monte Carlo es basa exclusivament en retorns reals d'episodis complets i no utilitza bootstrapping. En canvi, els mètodes TD actualitzen els Q-valors a partir d'estimacions pròpies del futur, fet que amplifica el soroll i pot destabilitzar el procés d'aprenentatge en contextos molt exploratoris o amb transicions lliscants.

4. Actualització

   S'utilitza l'enfocament First-Visit, que només actualitza els valors Q en la PRIMERA aparició de cada(estat, acció). Això evita donar massa pes als (s,a) que apareixen moltes vegades en un mateix episodi i redueix la variància.

5. Paràmetres inicials
   - γ: El factor de descompte controla com d'importants són les recompenses futures en el càlcul del retorn G_t. Un valor elevat de γ:
     - Incentiva que l'agent valori trajectòries que eventualment arriben a l'objectiu

    - Però també permet discriminar algunes trajectòries per ser massa llargues o arriscades
- ε: MonteCarlo necessita varietat de trajectòries completes, perqué el retorn G depén de tota la seqüència de moviments. Amb ε = 0.1:
    - el 10% de les accions són aleatòries
    - la política té prou exploració per visitar diferents trajectòries
    - però la resta del temps explota els valors de Q ja apresos
- Si ε fos massa baix, MonteCarlo podria quedar encallat en una trajectòria dolenta i no descobrir altres camins. D'altra banda, si ε fos massa alt, les trajectòries serien massa caòtiques i la mitjana tardaria més a estabilitzar-se.

Qué passaria si enlloc d'utilitzar ε = 0,1 utilitzassim ε = 0,2, per exemple? Llavors tendriem un 10% més d'aleatorietat, cosa que no faria cap mal al MonteCarlo, ja que el métode no fa bootstrapping i calcula els valors de Q a partir de retorns complets. De fet, ε = 0,2 ajudaria a generar més trajectòries diverses, però s'ha decidit seguir una política molt conservadora en aquest aspecte

Algoritme Genètic

A diferència de SARSA i Q-Learning, que aprenen estimant valors Q(s,a), l'algoritme Genètic realitza una cerca directa a l'espai de polítiques. No aprenem com de bó es un estat, sino que evolucionam directament el comportament final de l'agent.

Característiques del nostre Disseny

1. Representació directa (Genotip = Política) Cada "individu" de la població és, literalment, la taula de política: un vector de longitud 16 (els estats) on cada posició indica l'acció a prendre (0, 1, 2 o 3).

- En lloc d'actualitzar valors matemàtics, l'algoritme combina i muta directament aquests vectors d'accions.

2. Avaluació Estocàstica (Fitness mitjana) Com que l'entorn és relliscós (is_slippery=True), avaluar un individu una sola vegada no és fiable (podria guanyar per sort o perdre per mala sort).

- La solució implementada: La nostra funció evaluate_policy executa 50 episodis per individu i calcula la mitjana d'èxits.
- Això actua com un filtre de renou: les polítiques arriscades que passen prop dels forats tindran una variança alta i una mitjana baixa, mentre que les polítiques robustes i segures obtindran un fitness consistentment alt.

3. Mecanismes d'Evolució: Per generar noves solucions a partir de les existents, utilitzem tres operadors:

Selecció per Torneig (tournament_selection): Agafem 3 individus a l'atzar i seleccionem el millor. Això manté la diversitat i evita que un sol "super-individu" domini la població massa ràpid.

Encreuament d'un punt (one_point_crossover): Tallem la política de dos pares per un punt aleatori i intercanviem les cues. Això permet combinar, per exemple, un bon inici d'una política amb un bon final d'una altra.

Mutació (mutate): Amb una probabilitat del 10%, canviem una acció a l'atzar. Això és vital per introduir exploració i evitar quedar atrapats en òptims locals.

4. Manteniment de la millor solució: Al codi, best_individual es passa directament a la següent generació. Això garanteix que la millor solució trobada mai es perdi per culpa d'una mutació destructiva o mala sort en el torneig, assegurant que la qualitat de la població mai retrocedeix.

Programació dinàmica

Tot i que teòricament la Programació Dinàmica (DP) és capaç de resoldre entorns estocàstics mitjançant l'equació de Bellman, la seva aplicació pràctica requereix un requisit fonamental: disposar d'un model complet i a priori de l'entorn (Model-Based). Això implica conèixer explícitament la matriu de probabilitats de transició.

En el cas de FrozenLake amb la variable is_slippery a true, ens trobam amb la següent situació:

1. Accés al model (La variable oculta): Tècnicament, la llibreria Gymnasium permetria accedir a les probabilitats exactes de relliscada (33.3%) mitjançant l'atribut intern env.unwrapped.P. Si accedíssim a aquesta variable, podríem resoldre el problema amb Value Iteration sense cap dificultat matemàtica.
2. Restricció del problema (Enfocament Model-Free): No obstant això, l'objectiu d'aquest problema és simular un escenari real d'Aprenentatge per Reforç on l'agent no té accés al codi intern de l'univers. Tractam l'entorn com una "Caixa Negra".

Conclusió:

Atès que assumim que l'agent no pot consultar les probabilitats de transició ni la funció de recompensa directament del codi (ja que això seria equivalent a conèixer les regles físiques del món abans d'haver-hi entrat), no és possible calcular les esperances matemàtiques necessàries per als algorismes de Programació Dinàmica.

Aquesta absència d'informació sobre el model és el que ens obliga a emprar mètodes d'aprenentatge basats en la interacció i l'experiència, com ara Q-Learning o SARSA, que estimen la política òptima mitjançant assaig i error sense necessitat de conèixer les probabilitats de relliscada per avançat.

Polítiques Apreses

SARSA

Política final:

['←', '↑', '↓', '↑']

['←', '←', '↓', '←']

['↑', '↓', '←', '←']

['←', '→', '↓', '←']

- Política conservadora.
- Prioritza moviments segurs, és a dir, moviments que t'estampin contra la paret
- Evita zones amb risc → menys probabilitat d'arribar al Goal.
- Reflecteix aprenentatge on-policy (evita riscos en entrenament → evita riscos sempre).

  Política estable però poc òptima la qual opta per la seguretat i no per explorar moviments.

Q-Learning

Política final:

['↓', '↑', '←', '↑']

['←', '←', '→', '←']

['↑', '→', '↓', '←']

['←', '→', '↓', '←']

- Molta més direccionalitat que en SARSA. Q-learning intenta baixar i anar cap a la dreta: accions que aproximen al Goal si tot va bé.
- Alguns estats apunten clarament cap al camí òptim del tauler FrozenLake 4x4:
  - Cap avall (↓)
  - Cap a la dreta (→)
  - Ja que l'objectiu es troba a la casella (4,4)
- També s'observen zones amb decisions xareques

Política valenta i orientada al camí curt però afectada per l'alta estocasticitat del terreny lliscant

MonteCarlo

Política final (greedy):

['←', '↑', '→', '↑']

['↓', '←', '←', '←']

['↑', '↓', '←', '←']

['←', '→', '→', '←']

- MonteCarlo tendeix a tenir polítiques molt estables perquè no usa bootstrap: només actualitza Q al final dels episodis.
- El patró que mostra coincideix amb el comportament típic de MonteCarlo:
  - Moviments deterministes cap al Goal (sobretot les → i ↓ finals).
  - Decisions semblants a Q-learning però una mica més "uniformes".
- En alguns estats presenta moviments que coincideixen amb els camins reals que s'han vist durant episodis d'èxit.

Política consistent que depen dels episodis d'èxit observats

Algoritme Genètic

Política Final:

['←', '↑', '↑', '↑']

['←', '↑', '→', '↓']

['↑', '↓', '←', '↑']

['→', '→', '↓', '↓']

L'Algoritme Genètic mostra una política molt coherent i global, resultat del fet que opera sobre polítiques senceres en lloc d'actualitzar accions individuals. Es poden veure patrons clars (↑ a la zona superior, →→↓↓ al quadrant inferior) que reflecteixen un camí robust cap a l'objectiu.

Comparació de rendiments

Comparació temporal

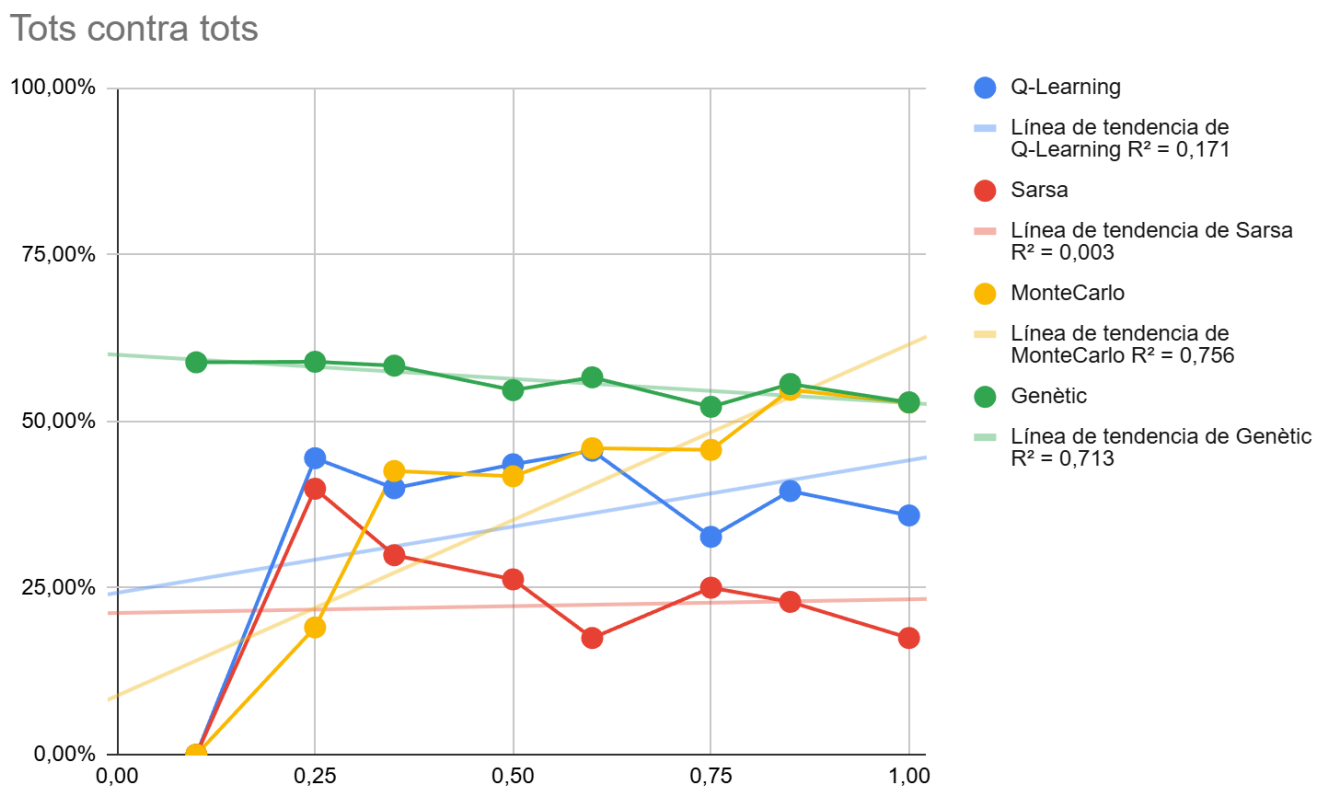
Execució	SARSA	Q-Learning	MonteCarlo	Genètic
Mitjana	59.5s	76.9s	59.75s	173,6s

Els temps d'execució obtinguts són coherents amb la naturalesa computacional de cada algoritme. SARSA i MonteCarlo presenten els temps més baixos perquè les seves actualitzacions són simples i es limiten a una operació per pas (SARSA) o una a final d'episodi (MonteCarlo). Q-Learning és el segon algoritme més costós en temps, ja que en cada pas ha de calcular el màxim de la funció Q i aplicar bootstrapping, cosa que incrementa el cost computacional.

L'algoritme genètic és més costós que la resta, ja que en lloc d'actualitzar una sola política de manera incremental, ha d'avaluar contínuament una població sencera d'individus, aplicar mutacions, creuaments i selecció a cada generació.

Les variacions puntuals observades entre execucions són normals en entorns Python i depenen de l'estat del sistema. En conjunt, tots els resultats són consistents i representatius del comportament esperat dels tres algoritmes.

Comparació gràfica



En aquesta secció es comparen els comportaments dels quatre algoritmes principals utilitzats: SARSA, Q-Learning, MonteCarlo i l'Algoritme Genètic, analitzant com afecta el nivell d'exploració (ε) al seu rendiment. Totes les proves s'han duit a terme amb 250.000 episodis, utilitzant γ = 0,9 i α = 0,1, per tal d'evitar variances excessives i obtenir una avaluació estable.

En primer lloc, **SARSA i Q-Learning** presenten comportaments marcadament diferents quan ε augmenta. Ambdós funcionen bé amb valors baixos d'exploració (ε = 0,1), però a mesura que incrementam ε, SARSA, com a algoritme on-policy, comença a desestabilitzar-se. El fet d'aprendre sobre la política real executada fa que, amb massa exploració, no aconsegueixi convergir cap a una política clara i el seu rendiment cau amb força. En canvi, Q-Learning, que és off-policy i es basa en bootstrapping, resisteix millor nivells alts d'exploració i manté una estabilitat acceptable, tot i que segueix obtenint els seus millors resultats amb ε baixos, quan pot explotar el càlcul del màxim Q(s,a) sense interferències d'exploració excessiva.

MonteCarlo mostra un patró oposat al d'aquests dos mètodes temporals. Com que no fa bootstrapping i necessita trajectòries completes per estimar els retorns, la seva qualitat

d'aprenentatge depèn molt positivament de l'exploració. Això es reflecteix clarament en la tendència ascendent dels resultats quan ε augmenta (R^2 al voltant de 0.71–0.75). Amb més exploració, MonteCarlo recull trajectòries molt variades que milloren l'estimació del valor de cada acció, arribant a superar Q-Learning quan ε és elevat i assolint percentatges d'encert que poden situar-se entre el 55% i el 66%.

L'Algoritme Genètic destaca clarament sobre la resta per la seva robustesa davant l'estocasticitat del problema. A diferència dels mètodes temporals, no depèn de l'elecció d'una acció concreta en un estat específic, sinó que optimitza estratègies completes mitjançant evolució poblacional.

L'elitisme garanteix que les millors polítiques sempre es preservin i la mutació introdueix variació sense desestabilitzar la política guanyadora. Aquest equilibri entre estabilitat i exploració fa que el seu rendiment sigui el més alt i, sobretot, el més estable, independentment del nivell d'exploració ε (factor de mutació).

En conjunt, la comparativa evidencia quatre perfils clarament diferenciats:

- SARSA: el que més pateix amb ε elevat; variància alta i degradació marcada.
- Q-Learning: estable amb exploració moderada, òptim amb ε baix.
- MonteCarlo: millora com més exploració hi ha; necessita molta diversitat de trajectòries.
- Genètic: el més sòlid i constant, independent del valor d'ε.

Aquest enfocament conjunt i compacte permet entendre de manera clara els punts forts i febles de cada tècnica i justifica la decisió de presentar tots els resultats en un únic apartat comparatiu, molt més net, visual i directe.

Conclusió

En conclusió, els experiments mostren que SARSA és l'algoritme més inestable en entorns lliscants amb exploració alta, mentre que Q-Learning només rendeix bé amb ε baixos. MonteCarlo millora clarament amb més exploració gràcies al fet que no fa bootstrapping, i l'algoritme Genètic és el més robust i consistent en tots els escenaris. En conjunt, cada mètode destaca en condicions diferents i això confirma la importància del balanç entre exploració i estratègia d'aprenentatge.